

Order Tracking of Mobius Propeller Data

The Matlab code for this problem is listed in Appendix 1.

Spectrogram for Channel 1

Figure 1 shows the spectrogram for the Channel 1 pressure recording at an angle of $+23^\circ$.

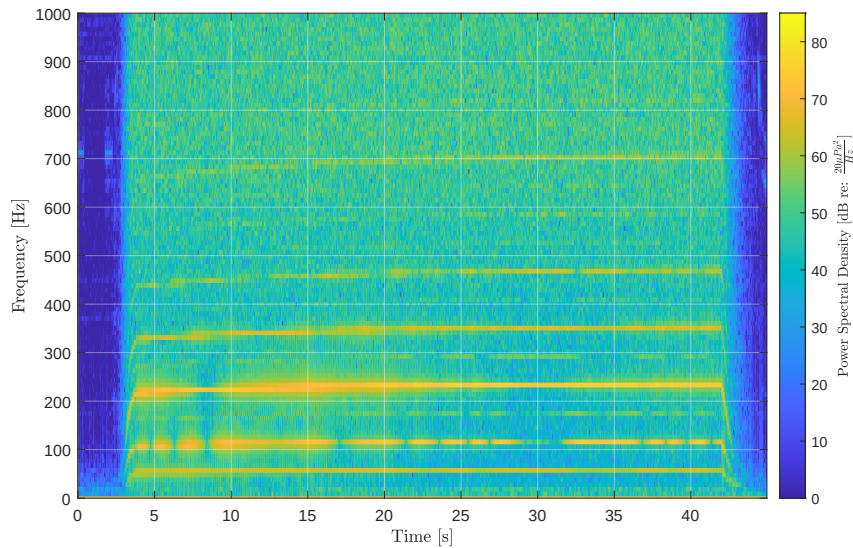


Figure 1: Spectrogram for the sound pressure recording at $+23^\circ$.

The bottom trace in the spectrogram, which corresponds to shaft rotation, is about 58.6 cycles per second.

Shaft Rotation Speed

Figure 2 shows the estimated rotation speed of the shaft, calculated on a per revolution basis.

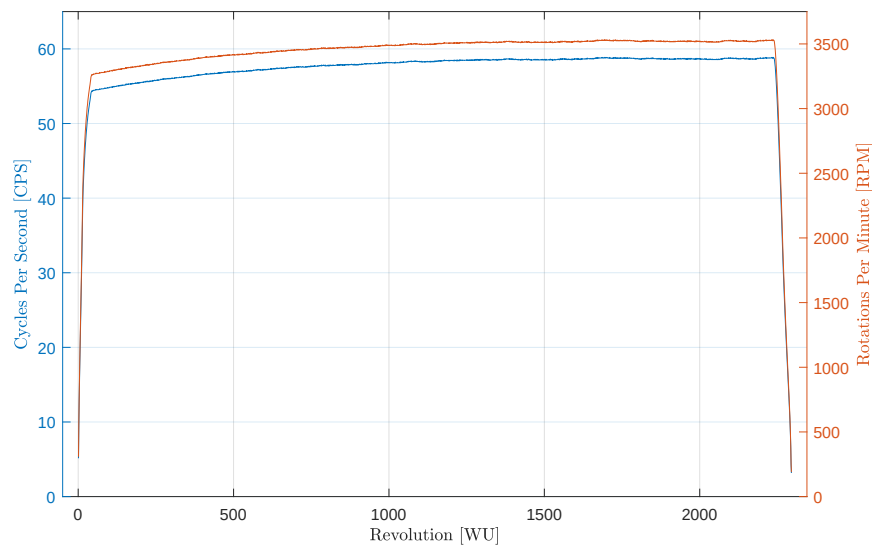


Figure 2: Estimation of shaft rotation speed per revolution.

The maximum shaft rotation speed is approximately 58.9 cycles per second, or 3,532 RPM.

Second-order Tracking for Channel 1

Figure 3 shows the second-order tracking profile for the Channel 1 pressure recording at $+23^\circ$.

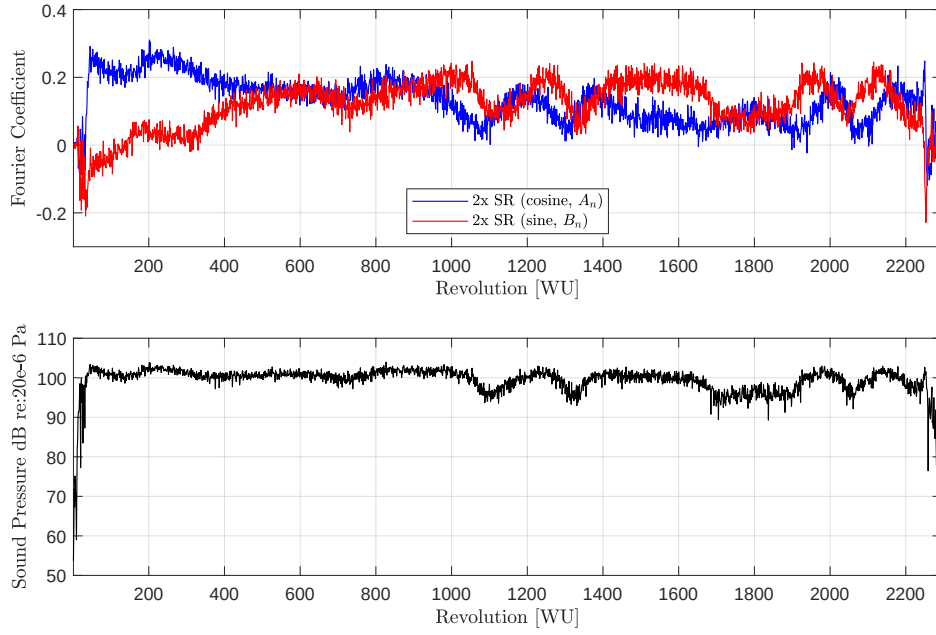


Figure 3: Second-order tracking for Channel 1.

Second-order Tracking Across Channels

Figure 4 shows the second-order tracking profiles for all of the pressure recording channels.

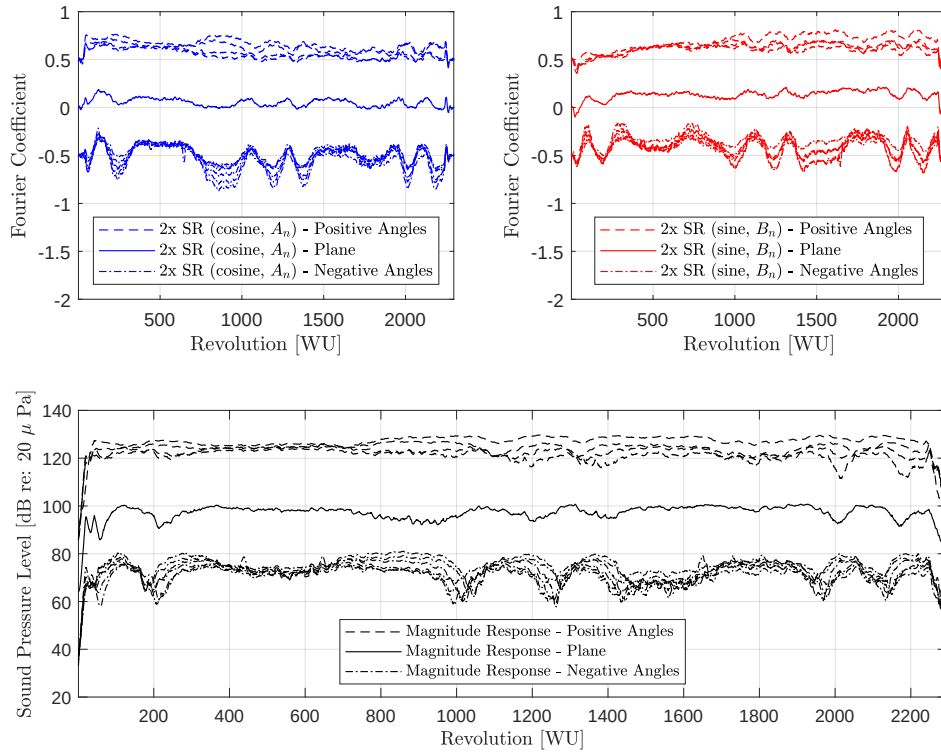


Figure 4: Second-order tracking across channels. A 0.5 offset was used to visually separate the Fourier coefficient profiles. A 20 dB offset was used to separate the sound pressure level profiles.

To visualize the tracking behaviour, the second-order tracking profiles were put into three groups: positive angle channels (1 to 4), a 0° channel (in-plane with the propeller), and negative angle channels (6 to 11). For the respective Fourier coefficients, an offset of 0.5 was added to the positive angle channels, no offset for the 0° channel, and an offset of -0.5 was added to the negative angle channels. For the sound pressure levels, an offset of 20 dB was used to offset the channel groups in the same fashion.

Based on these groups, there is a discernible difference between the sound level pressure profiles above and below the propeller. The profiles above the plane show less variation than the profiles below the plane of the propeller. These larger pressure variations below the plane of the propeller suggest that mobius propellers might make drones more audible and easier to track.

Fourth-order Tracking Across Channels

Figure 5 shows the fourth-order tracking profiles for all of the pressure recording channels.

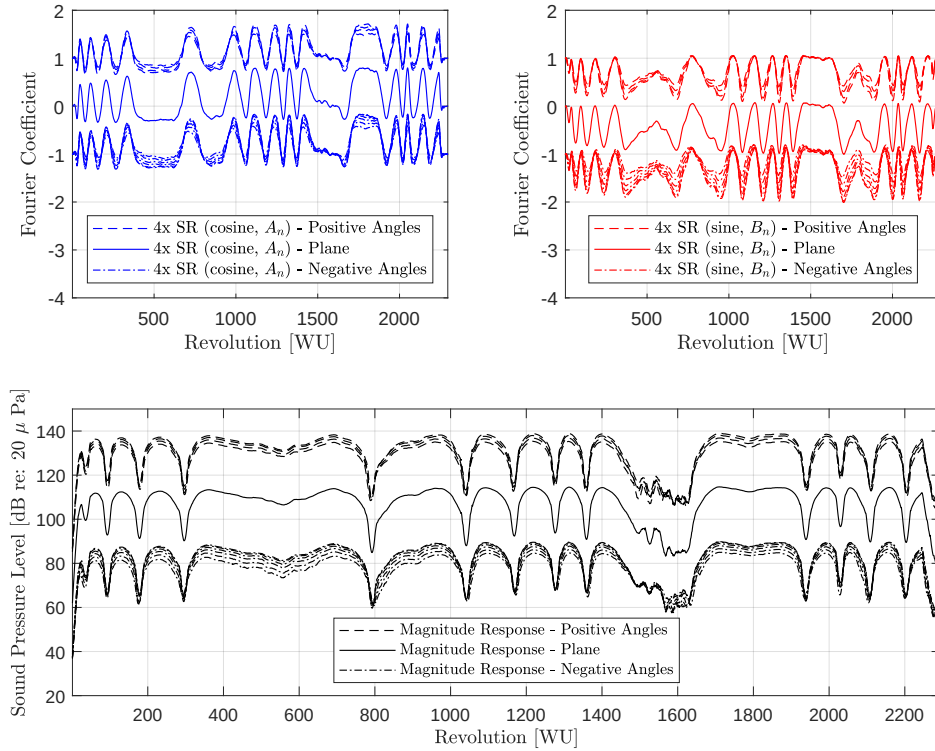


Figure 5: Fourth-order tracking across channels. A 1.0 offset was used to visually separate the Fourier coefficient profiles. A 20 dB offset was used to separate the sound pressure level profiles

As with the second-order case, offsets were used to offset the channel angle groups. For the respective Fourier coefficients, an offset of 1.0 was added to the positive angle channels, no offset for the 0° channel, and an offset of -1.0 was added to the negative angle channels. For the sound pressure levels, an offset of 20 dB was used to offset the channel groups in the same fashion.

Unlike the second-order case, where there is more sound pressure level variation below the plane of the propeller compared to above the propeller, there is large sound pressure variation below and above the propeller for fourth-order tracking.

1 Appendix - Matlab Code

```
1
2
3
4 %% Synopsis
5
6 % Problem 5 — Order Tracking of Mobius Propeller Data
7
8 % See Lecture 26
9 % "D:\15 Downloads\00 GitHub\ACS_547\35 Lectures\26 Monday, April 21, 2025\Lecture 26 — Shafting and
   bearings.pptx"
10
11
12
13 %% Environment
14
15 close all; clear; clc;
16 % restoredefaultpath;
17
18 addpath( genpath( './00 Support' ), '-begin' );
19
20 % set( groot, 'DefaultFigurePosition', [ 230 730 3*750 500 ] ); % x, y, width, height
21
22 set( 0, 'DefaultFigurePaperPositionMode', 'manual' );
23 set( 0, 'DefaultFigureWindowStyle', 'normal' );
24 set( 0, 'DefaultLineLineWidth', 0.6 );
25 set( 0, 'DefaultTextInterpreter', 'Latex' );
26
27 format ShortG;
28
29 color_map = slanCM( 'ColorBlind', 4 );
30
31 pause( 1 );
32
33
34
35 %% Load Data
36
37 load( 'mobius_prop_data.mat' ); % Variable(s): fs; mic_angles_degrees; p; trigger
38
39 % fs — Sample rate of 80 kHz (scalar).
40 % mic_angles_degrees — Microphone angles relative to horizontal plane (11-by-1; 23, 18, 6, 0, -6, -12,
   -18, -23, -28, and -32).
41 % p — Recorded pressures for the corresponding microphone angles (3,600,000-by-11).
42 % trigger — Shaft rotation synchronization signal (3,600,000-by-11).
43
44
45
46 %% Visualize Experimental Data
47
48 net_time = size( p, 1 ) / fs; % 45 seconds
49
50 time_indices = ( 0:1:( numel( trigger ) - 1 ) ) ./ fs;
51 time_indices = time_indices(:);
52
53
54 figure( 'Name', 'Trigger signal and Channel 1 Pressure Data' ); ...
55
56 h1 = subplot( 2, 1, 1 ); ...
57 plot( time_indices, trigger, 'Color', color_map( 1, : ) ); grid on;
58 xlabel( 'Time [s]' ); ylabel( 'Voltage [V]' ); title( 'Shaft Trigger Signal' );
59 ylim( [ -0.5 4 ] );
60 h2 = subplot( 2, 1, 2 ); ...
61 plot( time_indices, p( :, 1 ), 'Color', color_map( 2, : ) ); grid on;
62 xlabel( 'Time [s]' ); ylabel( 'Pressure [Pa]' ); title( 'Recorded Pressure at 23$^\circ$
   Elevation' );
63 ylim( [ -5 5.5 ] );
64
65 linkaxes( [ h1 h2 ], 'x' );
66 xlim( [ -5 50 ] );
67
68
69
70 %% Spectrogram
71
72 signal = p(:, 1);
73
74 frame_length = 8192;
75 frame_hop = floor( 0.20 * frame_length );
76
77 a_spectrogram = spectrogram_September_26_2023( signal, frame_length, frame_hop, fs, 0 );
78
79 figure( 'Name', 'Spectrogram of Channel 1 Pressure Signal' ); ...
80
81 imagesc( a_spectrogram.time_indices, a_spectrogram.Sxx.frequencies, 20*log10( sqrt(a_spectrogram.
   Sxx.spectrum)./20e-6 ), [ 0 85 ] );
82 labelColorbar( 'Power Spectral Density [dB re: $\frac{20}{\mu}$ Pa$^2$]{Hz}$' ); grid on;
83 colormap parula; % Option: Turbo
84 set( gca, 'GridColor', 'w', 'GridAlpha', 0.4 );
85 xlabel( 'Time [s]' ); ylabel( 'Frequency [Hz]' );
```

```

86     set( gca, 'ydir', 'normal' );
87     ylim( [ 0 1e3 ] );
88
89
90
91 %% Locate the Leading Edges of the Trigger Data
92
93 % Smooth data to reduce noise; supports finding the leading edges.
94 smoothed_trigger_data = [ 0; diff( smooth( trigger, 'moving', 5 ) ) ];
95
96 % Threshold smoothed data to located leading edges.
97 threshold = 0.5;
98 edge_indices = find( threshold < smoothed_trigger_data );
99 leading_edges = edge_indices( 1:2:end );
100
101 temp = diff( leading_edges );
102 [ indices_to_remove, indices ] = find( temp == 2 );
103 clear temp;
104
105 temp = leading_edges;
106 temp( indices_to_remove ) = [ ];
107
108 leading_edges = temp;
109 clear temp;
110
111 time_indices_leading_edge = leading_edges ./ fs;
112
113
114 % figure( 'Name', 'Revolution Segmentation' ); ...
115 %
116 %     plot( trigger( 1:1:end ), 'Color', 'k' ); hold on;
117 %
118 %     for index = 1:1:numel( leading_edges )
119 %         line( [ leading_edges( index ) leading_edges( index ) ], [ 0 4 ], 'Color', 'b' ); grid on;
120 %     end
121 %
122 %     axis( [ 1 numel( trigger ) -0.5 4.5 ] );
123
124
125
126 %% Segment Microphone Recording Channels
127
128 % Channel 1: 23 degrees
129 % Channel 2: 18 degrees
130 % Channel 3: 12 degrees
131 % Channel 4: 6 degrees
132 % Channel 5: 0 degrees
133 % Channel 6: -6 degrees
134 % Channel 7: -12 degrees
135 % Channel 8: -18 degrees
136 % Channel 9: -23 degrees
137 % Channel 10: -28 degrees
138 % Channel 11: -32 degrees
139
140
141 start_indices = 1:1:( numel( leading_edges ) - 1 ); end_indices = 2:1:numel( leading_edges );
142
143 frame_set_indices = [ leading_edges( start_indices(:) ) leading_edges( end_indices(:) ) ];
144 number_of_revolutions = size( frame_set_indices, 1 );
145
146
147 for channel_index = 1:1:size( p, 2 )
148
149     for frame_index = 1:1:size( frame_set_indices, 1 )
150
151         channel_segments{ frame_index, channel_index } = p( frame_set_indices( frame_index, 1 ):1:
152             frame_set_indices( frame_index, 2 ), channel_index );
153
154         slope_theta = (2*pi) ./ size( channel_segments{ frame_index, channel_index }, 1 );
155         theta{ frame_index, channel_index } = slope_theta .* ( 0:1:size( channel_segments{
156             frame_index, channel_index }, 1 ) - 1 );
157
158     end
159 end
160
161
162 %% Compute Instantaneous RPM
163
164 for channel_index = 1:1:size( p, 2 )
165     for frame_index = 1:1:size( frame_set_indices, 1 )
166         rpm_estimate{ frame_index, channel_index } = 60 * ( 1 / ( numel( channel_segments{ frame_index,
167             channel_index } ) ./ fs ) );
168     end
169 end
170
171 figure( 'Name', 'Shaft Speed' ); ...
172
173 yyaxis left; ...
174 plot( [ rpm_estimate{ :, 1 } ] ./ 60 ); grid on;

```

```

175     ylabel( 'Cycles Per Second [CPS]' );
176     ylim( [ 0 65 ] );
177
178     yyaxis right; ...
179     plot( [ rpm_estimate{ :, 1 } ] ); grid on;
180     ylabel( 'Rotations Per Minute [RPM]' );
181
182     xlabel( 'Revolution [WU]' );
183
184     axis( [ -50 number_of_revolutions+50 0 3.75e3 ] );
185     shg;
186
187
188
189 %% Compute Fourier Coefficients
190
191 An = nan( size( frame_set_indices, 1 ), 11 ); Bn = An;
192
193 TRACKING_ORDER = 2;
194 % TRACKING_ORDER = 4;
195
196 for channel_index = 1:size( p, 2 )
197
198     for frame_index = 1:size( frame_set_indices, 1 )
199
200         M = numel( channel_segments{ frame_index, channel_index } );
201
202         An( frame_index, channel_index ) = ...
203             2/M * channel_segments{ frame_index, channel_index }.' * cos( TRACKING_ORDER/2 .* theta{
204                 frame_index, channel_index } ).';
205
206         Bn( frame_index, channel_index ) = ...
207             2/M * channel_segments{ frame_index, channel_index }.' * sin( TRACKING_ORDER/2 .* theta{
208                 frame_index, channel_index } ).';
209
210     end
211 end
212
213
214 %% Plot Fourier Coefficients An and Bn and the Overall Sound Pressure Level for Channel 1
215
216 if ( TRACKING_ORDER == 2 )
217
218     figure( 'Name', 'Second-order Tracking and Associated Sound Pressure Level for Channel 1' ); ...
219
220     subplot( 2, 1, 1 ); ...
221     plot( An( :, 1 ), 'Color', 'b' ); hold on;
222     plot( Bn( :, 1 ), 'Color', 'r' ); grid on;
223     legend( '2x SR (cosine, $A_n$)', '2x SR (sine, $B_n$)', 'Location', 'South', '
224         Interpreter', 'Latex' );
225     xlabel( 'Revolution [WU]' ); ylabel( 'Fourier Coefficient' );
226     grid on; axis( [ 1 number_of_revolutions -0.3 0.4 ] );
227
228     subplot( 2, 1, 2 ); ...
229     plot( 10*log10( ( An( :, 1 ).^2 + Bn( :, 1 ).^2 ) ./ 2e-6^2 ), 'Color', 'k' ); grid on;
230     xlabel( 'Revolution [WU]' ); ylabel( 'Sound Pressure dB re:20e-6 Pa' );
231     axis( [ 1 number_of_revolutions 50 110 ] );
232
233     shg;
234
235 else
236
237     figure( 'Name', 'Fourth-order Tracking and Associated Sound Pressure Level for Channel 1' ); ...
238
239     subplot( 2, 1, 1 ); ...
240     plot( An( :, 1 ), 'Color', 'b' ); hold on;
241     plot( Bn( :, 1 ), 'Color', 'r' ); grid on;
242     legend( '4x SR (cosine, $A_n$)', '4x SR (sine, $B_n$)', 'Location', 'South', '
243         Interpreter', 'Latex' );
244     xlabel( 'Revolution [WU]' ); ylabel( 'Fourier Coefficient' );
245     grid on; axis( [ 1 number_of_revolutions -1 1 ] );
246
247     subplot( 2, 1, 2 ); ...
248     plot( 10*log10( ( An( :, 1 ).^2 + Bn( :, 1 ).^2 ) ./ 2e-6^2 ), 'Color', 'k' ); grid on;
249     xlabel( 'Revolution [WU]' ); ylabel( 'Sound Pressure dB re:20e-6 Pa' );
250     axis( [ 1 number_of_revolutions 50 120 ] );
251
252     shg;
253
254 end
255
256
257 %% Plot Fourier Coefficients An and Bn and the Overall Sound Pressure Level for all Channels
258
259 ORDER = 3; FRAMELEN = 25;
260
261 FOURIER_OFFSET = 0.5;
262 MAGNITUDE_OFFSET = 25;

```



```

331
332     end
333     %
334     legend( [ h_above h_plane h_below ], { 'Magnitude Response — Positive Angles', 'Magnitude
        Response — Plane', 'Magnitude Response — Negative Angles' }, 'Location', 'South', '
        Interpreter', 'Latex' );
335     xlabel( 'Revolution [WU]' ); ylabel( 'Sound Pressure Level [dB re: 20-6μPa]' );
336     grid on; axis( [ 1 number_of_revolutions 20 140 ] );
337
338     shg;
339
340 else
341
342     FOURIER_OFFSET = 1;
343
344     figure( 'Name', 'Fourth-order Tracking and Associated Sound Pressure Level for all Channels' ); ...
345
346     CHANNEL_INDEX = 11;
347
348     h1 = subplot( 2, 2, 1 ); ...
349
350     for channel_index = 1:1:CHANNEL_INDEX
351
352         if ( channel_index < 5 )
353             h_above = plot( sgolayfilt( An( :, channel_index ), ORDER, FRAMELEN ) +
                FOURIER_OFFSET, 'Color', 'b', 'LineStyle', '-' ); hold on;
354             % fprintf( 1, '\n%d — %3.1f', channel_index, mic_angles_degrees( channel_index
                ) );
355             elseif ( channel_index == 5 )
356                 h_plane = plot( sgolayfilt( An( :, channel_index ), ORDER, FRAMELEN ), 'Color', 'b'
                ); hold on;
357                 % fprintf( 1, '\n%d — %3.1f', channel_index, mic_angles_degrees( channel_index
                ) );
358             else
359                 h_below = plot( sgolayfilt( An( :, channel_index ), ORDER, FRAMELEN ) —
                FOURIER_OFFSET, 'Color', 'b', 'LineStyle', '-.' ); hold on;
360                 % fprintf( 1, '\n%d — %3.1f', channel_index, mic_angles_degrees( channel_index
                ) );
361             end
362
363         end
364         %
365         legend( [ h_above h_plane h_below ], { '4x SR (cosine, $A_n$) — Positive Angles', '4x SR (
            cosine, $A_n$) — Plane', '4x SR (cosine, $A_n$) — Negative Angles' }, 'Location', '
            South', 'Interpreter', 'Latex' );
366         xlabel( 'Revolution [WU]' ); ylabel( 'Fourier Coefficient' );
367         grid on; axis( [ 1 number_of_revolutions -4 2 ] );
368
369         % fprintf( 1, '\n\n' );
370
371         h2 = subplot( 2, 2, 2 ); ...
372
373         for channel_index = 1:1:CHANNEL_INDEX
374
375             if ( channel_index < 5 )
376                 h_above = plot( sgolayfilt( Bn( :, channel_index ), ORDER, FRAMELEN ) +
                FOURIER_OFFSET, 'Color', 'r', 'LineStyle', '-' ); hold on;
377                 % fprintf( 1, '\n%d — %3.1f', channel_index, mic_angles_degrees( channel_index
                ) );
378             elseif ( channel_index == 5 )
379                 h_plane = plot( sgolayfilt( Bn( :, channel_index ), ORDER, FRAMELEN ), 'Color', 'r'
                ); hold on;
380                 % fprintf( 1, '\n%d — %3.1f', channel_index, mic_angles_degrees( channel_index
                ) );
381             else
382                 h_below = plot( sgolayfilt( Bn( :, channel_index ), ORDER, FRAMELEN ) —
                FOURIER_OFFSET, 'Color', 'r', 'LineStyle', '-.' ); hold on;
383                 % fprintf( 1, '\n%d — %3.1f', channel_index, mic_angles_degrees( channel_index
                ) );
384             end
385
386         end
387         %
388         legend( [ h_above h_plane h_below ], { '4x SR (sine, $B_n$) — Positive Angles', '4x SR (
            sine, $B_n$) — Plane', '4x SR (sine, $B_n$) — Negative Angles' }, 'Location', 'South',
            'Interpreter', 'Latex' );
389         xlabel( 'Revolution [WU]' ); ylabel( 'Fourier Coefficient' );
390         grid on; axis( [ 1 number_of_revolutions -4 2 ] );
391
392         fprintf( 1, '\n\n' );
393
394         h3 = subplot( 2, 2, [ 3 4 ] ); ...
395
396         for channel_index = 1:1:CHANNEL_INDEX
397
398             if ( channel_index < 5 )
399                 h_above = plot( sgolayfilt( 10*log10( ( An( :, channel_index ).^2 + Bn( :,
                channel_index ).^2 ) ./ 2e-6^2 ) + MAGNITUDE_OFFSET, ORDER, FRAMELEN ), 'Color'
                , 'k', 'LineStyle', '-' ); hold on;
400                 fprintf( 1, '\n%d — %3.1f', channel_index, mic_angles_degrees( channel_index
                ) );
401             elseif ( channel_index == 5 )

```



```

402         h_plane = plot( sgolayfilt( 10*log10( ( An( :, channel_index ).^2 + Bn( :,
403             channel_index ).^2 ) ./ 2e-6^2 ), ORDER, FRAMELEN ), 'Color', 'k' ); hold on;
404         fprintf( 1, '\n%d - %3.1f', channel_index, mic_angles_degrees( channel_index )
405             );
406     else
407         h_below = plot( sgolayfilt( 10*log10( ( An( :, channel_index ).^2 + Bn( :,
408             channel_index ).^2 ) ./ 2e-6^2 ) - MAGNITUDE_OFFSET, ORDER, FRAMELEN ), 'Color'
409             , 'k', 'LineStyle', '-.' ); hold on;
410         fprintf( 1, '\n%d - %3.1f', channel_index, mic_angles_degrees( channel_index )
411             );
412     end
413     %
414     legend( [ h_above h_plane h_below ], { 'Magnitude Response - Positive Angles', 'Magnitude
415         Response - Plane', 'Magnitude Response - Negative Angles' }, 'Location', 'South', '
416         Interpreter', 'Latex' );
417     xlabel( 'Revolution [WU]' ); ylabel( 'Sound Pressure Level [dB re: 20-$\mu$ Pa]' );
418     grid on; axis( [ 1 number_of_revolutions 20 150 ] );
419
420     shg;
421
422 end
423
424 %% Clean-up
425
426 if ( ~isempty( findobj( 'Type', 'figure' ) ) )
427     monitors = get( 0, 'MonitorPositions' );
428     if ( size( monitors, 1 ) == 1 )
429         autoArrangeFigures( 3, 4, 1 );
430     elseif ( 1 < size( monitors, 1 ) )
431         autoArrangeFigures( 2, 2, 2 );
432     end
433
434 end
435
436 fprintf( 1, '\n\n*** Processing Complete ***\n\n' );
437
438 %% Reference(s)
439
440 % https://www.mathworks.com/help/signal/ref/ordertrack.html#bvaw5ra-1-x
441
442 % https://www.mathworks.com/matlabcentral/answers/1439884-evaluate-rising-edge-sample-of-a-signal

```